

Document number	P1434R0
Date	2019-01-21
Project	Programming Language C++, SG12 (Undefined and Unspecified Behavior)
Reply-to	Hal Finkel <hfinkel@anl.gov>
Authors	Hal Finkel <hfinkel@anl.gov>, Jens Gustedt <jens.gustedt@inria.fr>, Martin Uecker <Martin.Uecker@med.uni-goettingen.de>,

Discussing Pointer Provenance

There is ongoing work on a proposal for WG14 based on this POPL 2019 paper: [Exploring C Semantics and Pointer Provenance](#). The authors of this paper, along with significant work by Jens Gustedt, are working on proposed wording changes to the C specification. Of the options discussed in that paper, the model variant currently receiving this attention is the provenance-not-via-integer, tainting all, user-disambiguation model (PNVI-taint-all-udis).

See also the storage-instance paper by Jens (WG14 [N2328](#)), and the closely-related formal model by [Kang et al. \(alt\)](#).

What follows is a summary of this model by Jens and Martin. This represents work still under development and active revision; early feedback from WG21 is requested.

A "storage instance" is the "byte array" that is created when either an object starts its lifetime (for static, automatic and thread storage duration) or an allocation function is called (`malloc`, `calloc` etc). Storage instances are more than just an address, they have a unique ID throughout the whole execution. Once their lifetime ends, another storage instance may receive the same address, but never the same ID.

The provenance of a valid pointer is the "storage instance" to which the pointer refers (or one past). This is part of the "abstract state" in C's abstract machine, not necessarily part of the object representation of the pointer itself.

Valid pointers keep provenance to the encapsulating storage instance of the referred object. When the storage instance dies (falls out of scope, end of thread, `free`) the pointer becomes indeterminate.

Ordered comparisons (`<`, `>`, `>=`, `<=`) between pointers are only defined when the two pointers have the same provenance. They then can be defined by the relative byte position in the byte array of the common storage instance.

Equality of pointers is handled by a case analysis:

- if both are null, they are equal
- if one is valid and the other is null, they are unequal
- if the types are function pointers, they compare equal if and only if the refereed functions are equal
- if both are valid, have different provenance and compare equal, then one is the end address of one storage instance and the other is the start address of another storage instance that happens to follow immediately in the address space
- if both are valid and have different provenance, they compare not equal
- if both are valid and have the same provenance they compare equal if and only if they have the same position in the byte array
- in all other cases (that is one of the pointers is indeterminate) the behavior is undefined

Pointer arithmetic (addition or subtraction of integers) preserves provenance. The pointer becomes indeterminate if the result is outside the storage instance or goes beyond the array that the pointer is referring to (or is the "one past" address).

Pointer difference is only defined for pointers with the same provenance and within the same array.

Pointer values can be copied by the usual means that is: assignment, `memcpy` and byte-wise copy. These copy over provenance in addition to the representation and the effective type. (There is certainly more work to do here to say exactly what that means. For the moment, let's go with "any copy operation that would propagate the effective type".)

No other manipulation of the representation of a pointer will lead to a valid pointer value, because neither the effective type nor the provenance can be reconstructed from such manipulations. Thus the value of such pointers is indeterminate.

A storage instance is "tainted" once any valid pointer with this provenance is converted to integer (cast) or to IO (`printf` with `"%p"`). For the sake of the "happened before" relation, "tainting" constitutes a side effect, even though the taint is not observable.

This "tainting" does *also* happen for the end address of a storage instance. An pointer-to-integer cast has to result in the same integer value, regardless if a the pointer has the provenance as end address of one storage instance A or as the start address of another storage instance B, where B happens to immediately follow A in the address space.

The idea behind "tainting" is that once a pointer has escaped to an integer or to IO, all aliasing analysis is jeopardized. On the other hand, pointers to a storage instance for which a compiler can prove that it is untainted (e.g a because it is stack variable and no address has been taken), can never alias unexpectedly.

An integer-to-pointer conversion (cast) or IO (`scanf` with `"%p"`) is only defined if the corresponding storage instance had been tainted, and if the result is a pointer to a byte (or one-after) of the storage instance.

Ambiguous Provenance:

With the above, there is one special case where a back-converted pointer (let's just assume integer-to-pointer) could have two different provenances. This can happen when:

- X is the end address (one past) pointer of a storage instance A and the start address of another storage instance B
- both storage instances A and B are tainted, that is at some point we did a pointer-to-integer conversion with two pointers, a having provenance A, and b having provenance B.

In such a situation, both A and B could be valid choices for the provenance.

Our trick is to leave which of A or B is chosen to the programmer. It is their responsibility to be consistent, and to disambiguate such situations when necessary:

If `p` is the result of an integer-to-pointer cast with two possible provenances and `p` is used with both provenances, the behavior is undefined.

Note: If the result `p` of an integer-to-pointer conversion is the end address of a tainted storage instance A and the start address of another tainted storage instance B that happens to follow immediately in the address space, a conforming program must only use one of these provenances in any expressions that is derived from `p`.

The following three cases determine if `p` is used with one of A or B and must hence not be used otherwise:

- Operations that constitute a use of `p` with either A or B and do not prohibit a use with the other:

- any relational operator or pointer subtraction where the other operand q may have both provenances, that is where q is also the result of a similar conversion and where $p == q$;
- $q == p$ and $q != p$ regardless of the provenance of q ;
- addition or subtraction of the value 0 ;
- conversion to integer.

For the latter, A and B must have been tainted before, and so a any choice of provenance, that would otherwise have tainted one of the storage instances, is consistent with any other use.

- Operations that, if otherwise well defined, constitute a use of p with A and prohibit any use with B :
 - Any relational operator or pointer subtraction where the other operand q has provenance A and cannot have provenance B .
 - $p + n$ and $p[n]$, where n is an integer strictly less than 0 .
 - $p - n$, where n is an integer strictly greater than 0 .
- Operations that, if otherwise well defined, constitute a use of p with B and prohibit any use with A :
 - Any relational operator or pointer subtraction where the other operand q has provenance B and cannot have provenance A .
 - $p + n$ and $p[n]$, where n is an integer strictly greater than 0 .
 - $p - n$, where n is an integer strictly less than 0 .
 - operations that access an object in B , that is indirection ($*p$ or $p[n]$ for $n == 0$) and member access ($p->member$).